MIPHI · POWERED BY PHISON

LibreChat with aiDAPTIV+

A private chat assistant that runs on your own hardware and remembers what it has already read.

User Manual and Setup Guide

01

Why it matters

What LibreChat and aiDAPTIV+ do, and what your business gets out of them.

02

Setting it up

Step by step instructions for running LibreChat with aiDAPTIV+ on your own hardware.

03

Checking it works

How to confirm on your own system that aiDAPTIV+ is saving time.

Version 1.0 · October 2026 · Prepared by MiPhi

Contents

Why LibreChat with aiDAPTIV+.....3

A typical example.....3
What LibreChat is.....3
What aiDAPTIV+ adds.....4
What happens when someone asks a question.....4
Benefits.....4
Test results.....5
Where it helps most.....6
How the setup is organised.....7

Part 1: Before You Begin.....8

Setting up Docker Desktop.....8
Cloning our aiDAPTIV-LibreChat Repository.....8
Default ports.....9
Adding the Admin Panel session secret to your .env.....9

Part 2: The aiDAPTIV Cache Configuration.....11

Part 3: Connecting LibreChat to Your LLM Model.....12

Add the custom endpoint in the Admin Panel.....12
Check the endpoint in LibreChat.....17

Part 4: Getting Started.....18

Part 5: Verifying the aiDAPTIV KV-Cache Offload Is Working.....19

Check the cache folder on disk.....19
Confirm a cache hit.....20

Part 6: Benchmark Comparison: TTFT, TPS & Cache Hit.....20

Run 2: Cache Hit (KV Reuse).....20

Why LibreChat with aiDAPTIV+

Most teams we speak to would like to use AI on their own documents, but two things hold them back. The first is privacy. The well-known AI chat tools run in someone else's cloud, so every document you paste in has to be sent there first. The second is speed. Give an AI a long document and you can wait several minutes before the first word of the answer shows up.

LibreChat with aiDAPTIV+ was put together to deal with both. This chapter explains how, in plain terms. If you would rather go straight to the installation, skip ahead to the Setup Guide.

A typical example

Say one of your engineers pastes a 25-page tender document into the chat and asks for the key delivery dates. In our tests, a document of that size kept the user waiting more than four minutes before anything appeared, because the AI had to read every page first.

Then a colleague asks about the payment terms in the same document. Without aiDAPTIV+, the AI reads all of it again and the wait starts over.

With aiDAPTIV+, the AI keeps what it learned the first time. In the same test the follow-up question started answering in **about a second and a half**, even though the server had been restarted in between. And the document never left your own machine.

What LibreChat is

LibreChat is an open-source chat app that looks and works much like ChatGPT. You host it yourself, so it runs on your own hardware and talks to an AI model you choose. Each person signs in with their own account and keeps their own conversation history.

A familiar chat screen

Anyone who has used ChatGPT already knows how to use LibreChat. There is nothing new to learn.

Questions about your files

Attach a document and ask about it, get a summary, or have it rewritten for a different audience.

Several models in one place

LibreChat can connect to more than one AI model, and people pick the one they want from a menu above the chat.

Admin Panel included

Your administrator adds models and changes settings from a web page, without editing files on the server.

What aiDAPTIV+ adds

When an AI model reads a document, it builds up a set of working notes about the text. Engineers call these notes the **KV cache**. They are normally kept in the graphics card's memory, which is small and expensive, so they get thrown away when space runs out or the server restarts. The next question then has to start from the beginning.

aiDAPTIV+, made by Phison, keeps those notes on a fast SSD instead. There is far more room there, and nothing gets thrown away. When a new question involves material the AI has already read, it loads its notes and carries on.

> **A SIMPLE WAY TO THINK ABOUT IT**
>
> Picture a librarian who answers questions about a book. Without aiDAPTIV+, they read the whole book again every time someone asks something. With aiDAPTIV+, they keep their notes on the shelf and can answer the next person almost straight away, even if the library closed overnight in between.

What happens when someone asks a question

Your users never need to think about any of this, but if you are curious, this is the sequence:

1
You ask
Someone types a question in LibreChat in their browser, with or without a file attached.

→

2
LibreChat passes it on
LibreChat sends the question and any documents to the AI model on your own hardware.

→

3
aiDAPTIV+ checks its notes
Before reading anything again, aiDAPTIV+ looks on the SSD for notes it already has.

→

4
The answer comes back
The reply starts to appear, and any new notes are saved for next time.

Benefits

Benefit	In practice
Your data stays in-house	The model, your documents and your chat history all stay on your own hardware. Nothing is sent to an outside AI provider.
Faster follow-up answers	Follow-up and repeat questions about the same documents reuse earlier work, so they start in seconds rather than minutes.

Benefit	In practice
Quicker even the first time	In our tests, the very first read of a new document was also about twice as fast with aiDAPTIV+.
Survives restarts	The notes sit on the SSD rather than in temporary memory, so the AI still remembers what it has read after the server restarts.
Better use of existing hardware	Long documents and long conversations no longer have to fit in the graphics card's memory, because the SSD takes the overflow.
Predictable cost	There are no per-question or per-token cloud charges. The only limit on usage is your own hardware.
Already configured	Our repository includes a ready-made librechat.yaml with the aiDAPTIV+ settings, which you import into the Admin Panel in a few clicks.
Suits teams	Everyone signs in with their own account and keeps their own chat history, and administrators decide which models are available.

Test results

We sent the same three prompts through LibreChat on the same PC, once without aiDAPTIV+ and twice with it: a first read, and then a second run after restarting the server. Part 5 of this guide shows you how to check the same things on your own system.

1.5 s

Time to the first word on a 12,000-token document after a server restart. Without aiDAPTIV+ the same prompt took 4 minutes 12 seconds.

156 to 187×

How much sooner the answer started across our three tests when aiDAPTIV+ reused its saved notes.

About 2×

How much faster the very first read of a new document was with aiDAPTIV+ in the same tests.

Time to the first word of the answer

Prompt size	Without aiDAPTIV+	aiDAPTIV+, first read	aiDAPTIV+, after restart
About 8,000 tokens 7,481 tokens	2 min 29 s	1 min 15 s	**0.8 s**
About 10,000 tokens 9,318 tokens	3 min 15 s	1 min 26 s	**1.2 s**
About 13,000 tokens 12,080 tokens	4 min 12 s	2 min 11 s	**1.5 s**

Measured by MiPhi in September 2026 using the gemma-4-26B-A4B model (4-bit) on an Intel Core Ultra 7 PC with integrated graphics. The aiDAPTIV+ runs used the aiDAPTIV+ middleware with an NVMe SSD cache, and the server was restarted between the first and second run. Your results will depend on your hardware, the model and the size of your documents.

WHAT THIS MEANS DAY TO DAY

Without aiDAPTIV+, the largest test kept the user waiting more than four minutes before anything appeared. With aiDAPTIV+, the follow-up started answering in about a second and a half. That is the difference between giving up and going back to your email, and simply reading the answer as it comes in.

Where it helps most

Support and service teams answer questions from product manuals and troubleshooting guides all day.

Legal and compliance teams work with contracts and policies that cannot leave the company.

Engineering and R&D teams handle specifications, design documents and test reports that are too sensitive for a public cloud.

Sales and bid teams work through long tenders and proposals where the same document gets questioned from every angle.

Regulated industries such as finance, healthcare and the public sector often need data to stay on their own premises.

How the setup is organised

The Setup Guide takes you from a fresh machine to your first answer. It is split into six parts:

Part	What you do	Result
Part 1 **Before You Begin**	Install Docker, download our LibreChat package, create your settings files and start the AI model server.	Everything LibreChat needs is in place.
Part 2 **The aiDAPTIV Cache Configuration**	Read what the two aiDAPTIV+ settings in librechat.yaml do.	Nothing to edit, just background.
Part 3 **Connecting LibreChat to Your Model**	Add your AI model in the Admin Panel, either by importing librechat.yaml or by filling in a short form.	LibreChat can talk to your model.
Part 4 **Getting Started**	Open the chat, pick your model and ask a question.	Your first answer.
Part 5 **Verifying the Cache**	Look at the SSD cache folder and the server log.	Confirmation that aiDAPTIV+ is saving and reusing its notes.
Part 6 **Benchmark**	Check the time to first token when saved work is reused.	Figures you can share with your team.

When you are ready, turn the page and start with Part 1.

SETUP GUIDE

Getting it running

LibreChat runs as a set of Docker Compose services and connects to an LLM model served separately by aiDAPTIV middleware. This manual uses our repository, which already includes the aiDAPTIV KV-cache settings integrated to LibreChat.

Part 1: Before You Begin

Setting up Docker Desktop

If you don't already have Docker installed, get Docker Desktop first, everything else in this manual depends on it.

1. Download Docker Desktop from the official site: https://www.docker.com/products/docker-desktop/
2. Run the installer and accept the defaults. On Windows, make sure the WSL2 option is selected if prompted. This is Docker's recommended backend.
3. Restart your machine if the installer asks you to.
4. Open Docker Desktop and wait until it says "Engine running" (a green status in the bottom-left corner) before continuing.
5. Confirm it from a terminal:

Open and run this command in a separate new terminal

```
PS C:\Users\aidap> docker --version
Docker version 29.8.0, build 88096ef
PS C:\Users\aidap> docker compose version
Docker Compose version v5.5.1
PS C:\Users\aidap>
```

Cloning our aiDAPTIV-LibreChat Repository

Use our aiDAPTIV-Integration-LibreChat repository. The miphi-dev branch already includes a pre-configured librechat.example.yaml with the aiDAPTIV cache settings, so you only need to add your own .env file before starting the stack. You copy librechat.example.yaml to librechat.yaml and upload librechat.yaml into the Admin Panel in step 5 below to add the aiDAPTIV endpoint to LibreChat.

github.com/Phison-MiPhi-Collaboration/aiDAPTIV-Integration-LibreChat (miphi-dev branch)

1. Clone the repository and switch into it:

```
git clone -b miphi-dev https://github.com/Phison-MiPhi-Collaboration/aiDAPTIV-Integration-LibreChat librechat
cd librechat
```

2. Create your environment file. LibreChat won't start without a .env. It holds the MongoDB connection string, session/JWT secrets, and any API keys. So copy the example that ships in the repo, then add the Admin Panel session secret to it before step 3 (see "Adding the Admin Panel session secret" below):

```
cp .env.example .env
# Windows: copy .env.example .env
```

3. Create your own librechat.yaml. LibreChat ships a template file called librechat.example.yaml. Copying it gives you a real librechat.yaml that LibreChat will actually read:

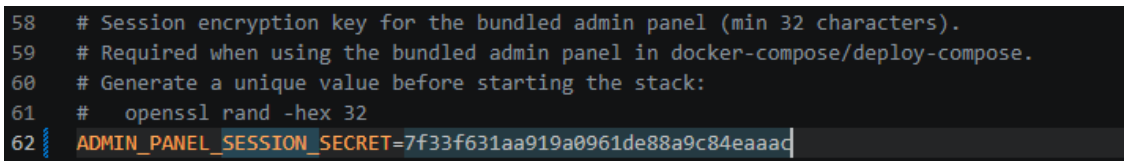
```
cp librechat.example.yaml librechat.yaml
# Windows: copy librechat.example.yaml librechat.yaml
```

Default ports

Service	Default port and address
LibreChat (chat UI)	3080 (http://localhost:3080)
LibreChat Admin Panel	3000 (http://localhost:3000)
aiDAPTIV llama.cpp server	8080 (http://127.0.0.1:8080)

Adding the Admin Panel session secret to your .env

The Admin Panel is bundled with LibreChat and will not start without a session secret. This is a random string of at least 32 characters that it uses to encrypt login sessions. You set it in your .env file before starting the stack.



The Admin Panel block in .env. In this example the setting is on line 62 (your line number may differ).

1. Open your .env file in any text editor and find the comment "Session encryption key for the bundled admin panel". The ADMIN_PANEL_SESSION_SECRET line sits directly under it (line 62 in the example above).

2. Choose a value. Either generate your own (recommended) or use the default value in step 3.

To generate your own, open a terminal and run the command below (macOS, Linux, WSL, or Git Bash on Windows). It prints a 64-character random string. Copy it:

```
openssl rand -hex 32
```

On Windows PowerShell, if openssl isn't available:

```
$b = New-Object byte[] 32
[Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($b)
($b | ForEach-Object { $_.ToString("x2") }) -join ""
```

3. Put the value on the ADMIN_PANEL_SESSION_SECRET line, with no spaces and no quotes. To use the default value instead, use exactly this line:

```
ADMIN_PANEL_SESSION_SECRET=7f33f631aa919a0961de88a9c84eaaac
```

4. Save .env, then start the stack (or restart it if it was already running):

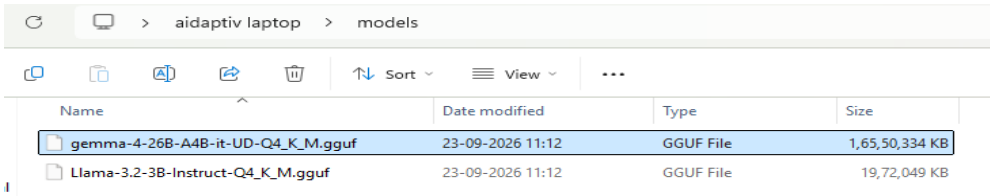
```
docker compose down
docker compose up -d
```

> **NOTE**
> The default value is fine for a local test on your own machine. Anyone who has this manual knows it, so generate your own before LibreChat is reachable by other users.

The rest of these prerequisites should also be in place before you start LibreChat:

6. Your LLM model file (a .gguf file) should be saved on the disk that your aiDAPTIV middleware can reach.

7. The aiDAPTIV middleware is installed as per its own setup guide, and the aiDAPTIV NVMe cache path it uses is configured within the middleware.



8. Start the LLM model server (via the aiDAPTIV middleware) pointed at that models folder, and confirm it reports a listening, OpenAI-compatible address, e.g. **http://127.0.0.1:8080/v1**. LibreChat will connect to this address in Part 3.

> **NOTE**
>
> LibreChat itself does not load the model. It talks to whatever OpenAI-compatible endpoint your aiDAPTIV Middleware exposes. Get that server running and confirmed first.

Part 2: The aiDAPTIV Cache Configuration

This section explains the two aiDAPTIV cache parameters. They live in the librechat.yaml file shown below (already present in our repository), which you upload to the Admin Panel in Part 3. You can also type the same two parameters in by hand.

1. Inside the custom endpoint block of librechat.yaml, the addParams: section already contains the two parameters required to offload and reuse the KV cache:

librechat.yaml

```yaml
version: 1.3.5
cache: true

endpoints:
  custom:
    - name: "aiDAPTIV+"
      apiKey: "no-key"
      baseURL: "http://host.docker.internal:8080/v1"

      models:
        default:
          - "gemma-4"
        fetch: false

      titleConvo: true
      titleModel: "gemma-4"
      titleMethod: "completion"
      titleTiming: "immediate"
      modelDisplayLabel: "aiDAPTIV+"

      addParams:
        cache_prompt: true
        offload_folder_name: "benchmark"
```

What the two keys do:

Key	What it does
cache_prompt	Tells the llama-server to save the KV cache generated during prefill instead of discarding it once the reply is sent, and to check for a matching cache before recomputing on the next request.
offload_folder_name	The subfolder name, under your aiDAPTIV NVMe cache root, where this model's KV-cache files are written (in this example, "benchmark-10"). You will see this exact name again in Part 5 when you go looking for the cache files on disk.

Because these are OpenAI-compatible request parameters, addParams (shown as Additional parameters in the Admin Panel) merges them directly into the request body LibreChat sends to your aiDAPTIV llama.cpp server on every completion request.

2. If you ever need a different offload_folder_name (e.g. to keep separate caches per model or benchmark run), change it under the endpoint's Additional parameters in the Admin Panel or edit that one line in your librechat.yaml and upload it again:

> **NOTE**
>
> With our repository the librechat.yaml already contains these settings, so you can go straight from Part 1 to Part 3 and upload it. This Part is here for reference, not as a required editing step.

Part 3: Connecting LibreChat to Your LLM Model

With the model server running (Part 1), add the aiDAPTIV llama.cpp server to LibreChat as a custom endpoint from the Admin Panel. There are two ways to do it: upload your librechat.yaml (fastest) or enter the values manually. Afterwards, check the endpoint in the LibreChat chat UI.

> **NOTE**
>
> The first account you register becomes the admin account. There is no default login. This is also the account you use to sign in to the Admin Panel (http://localhost:3000).

Add the custom endpoint in the Admin Panel

The aiDAPTIV llama.cpp server is added as a custom endpoint from the Admin Panel. It is the same OpenAI-compatible API on port 8080.

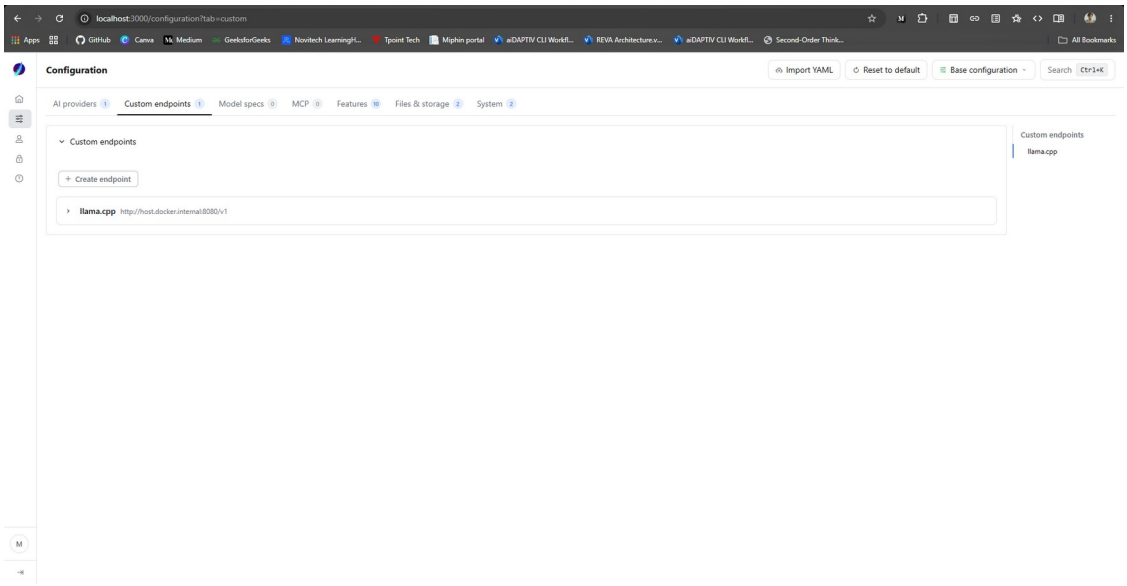
Users should first register an account in http://localhost:3080 and then access the Admin Panel at http://localhost:3000 .

Open the Admin Panel at http://localhost:3000 (a separate address from the LibreChat chat UI on port 3080), sign in with your admin account, and choose Configuration (the sliders icon in the left sidebar). Then use either Option 1 or Option 2 below.

Option 1: Upload your librechat.yaml (fastest)

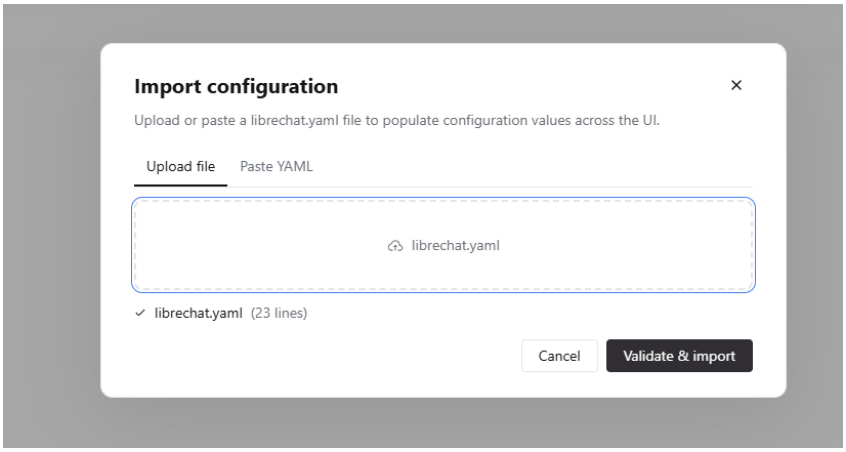
The Admin Panel can import a librechat.yaml file, for example the one from our repository (the same block shown in Part 2), so nothing needs to be retyped.

1. Go to Configuration → Custom endpoints and click Import YAML at the top right.



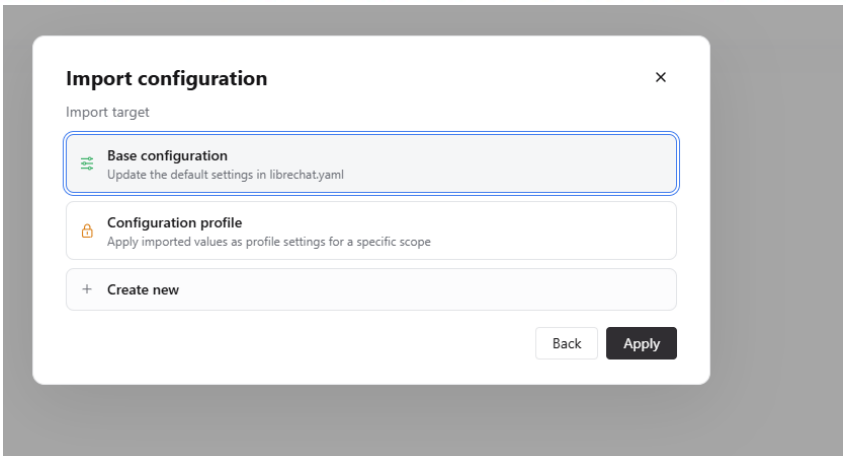
Configuration → Custom endpoints, with the Import YAML button at the top right

2. In the Import configuration window, use the Upload file tab to select your librechat.yaml (or open the Paste YAML tab and paste its contents). The file name and line count appear once it is loaded. Then click Validate & import.



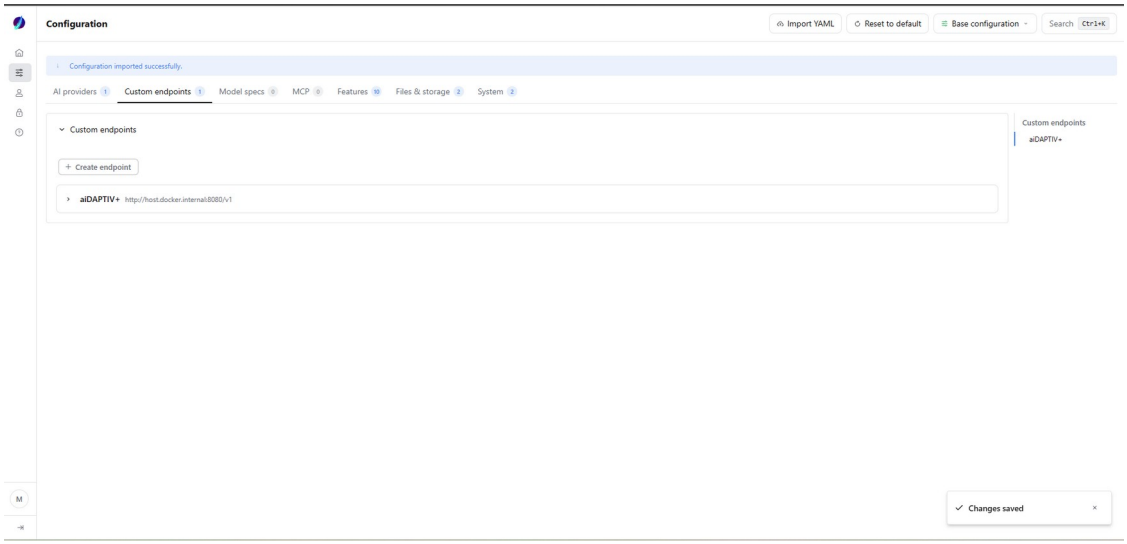
Import configuration: upload or paste librechat.yaml, then Validate & import

3. Choose the import target. Select Base configuration (updates the default settings) and click Apply.



Import target: choose Base configuration, then Apply

4. A "Configuration imported successfully" banner and a "Changes saved" message appear, and the endpoint (aiDAPTIV in this example) is listed under Custom endpoints with its base URL.



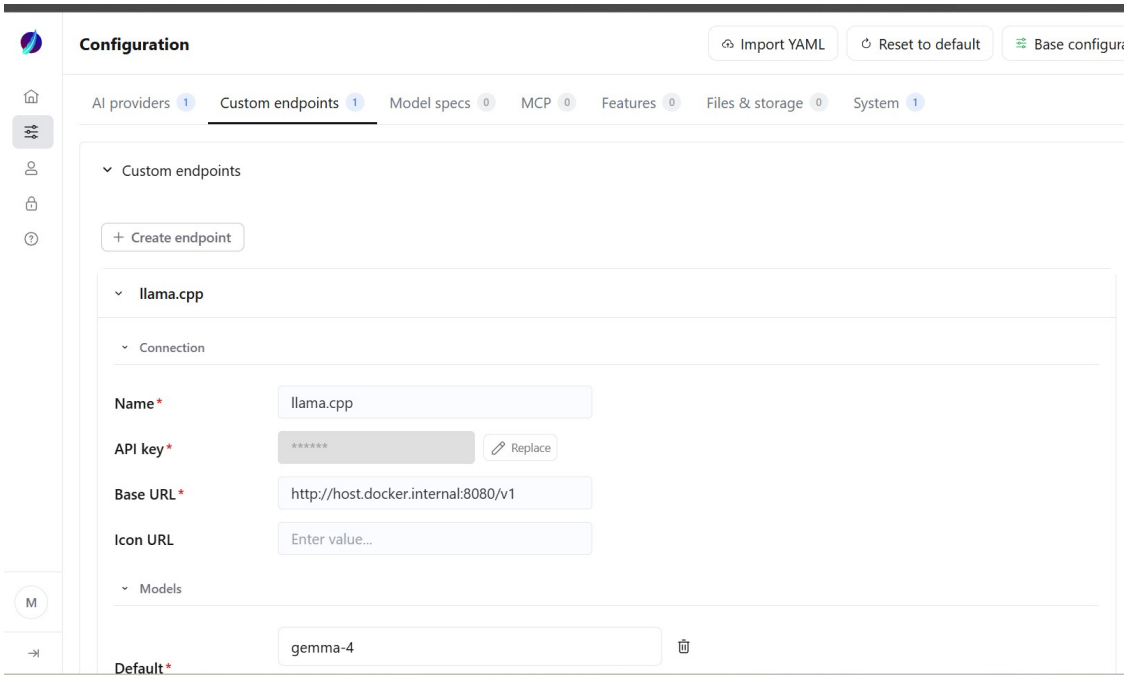
Import complete: the endpoint now appears under Custom endpoints

5. Expand the endpoint and check Advanced → Additional parameters: cache_prompt and offload_folder_name should be listed, as shown in Option 2 below.

Option 2: Enter the values manually

Use this if you don't have a librechat.yaml to import, or you prefer to fill in the form yourself.

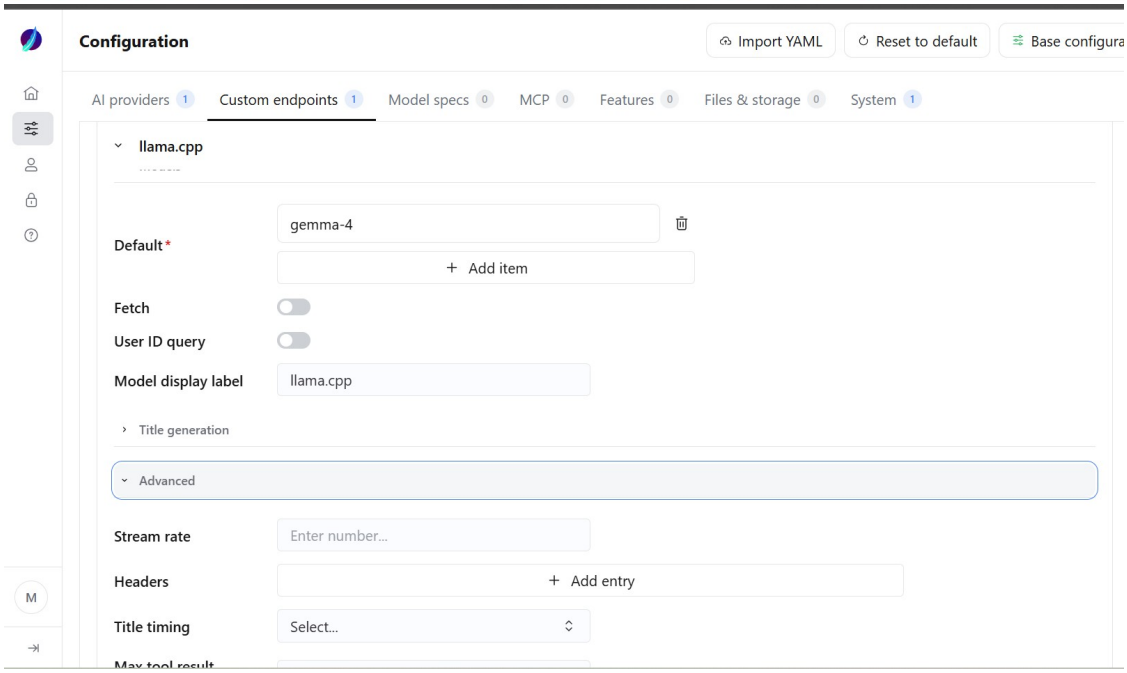
1. Under Configuration → Custom endpoints, click Create endpoint. Under Connection, fill in the fields below:



Configuration → Custom endpoints → Connection: Name, API key, Base URL and Models

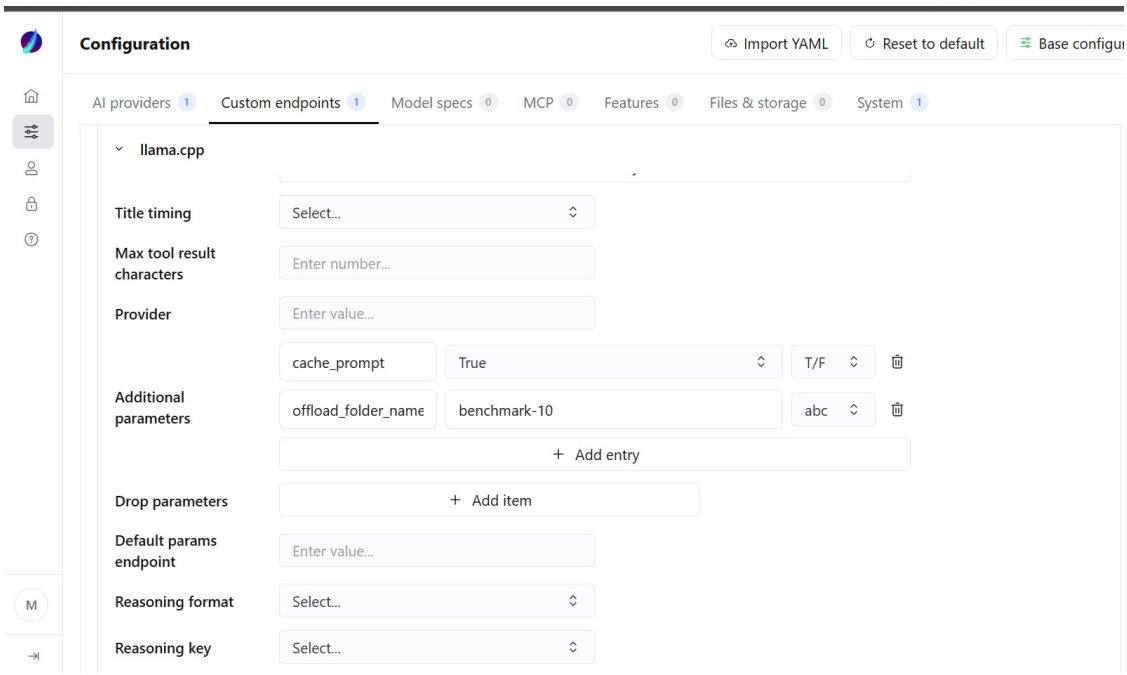
Field	What to enter
Name	A label for the endpoint, llama.cpp in this example. This is the name you will pick in the endpoint selector.
API key	A placeholder value unless your llama-server requires one. Once saved it is shown masked (******); use Replace to change it.
Base URL	http://host.docker.internal:8080/v1. Use host.docker.internal, not localhost or 127.0.0.1, because LibreChat runs in a container. Port 8080 is the aiDAPTIV llama.cpp server.
Icon URL	leave empty.

2. Under Models, click Add item and enter your model name exactly as your llama-server serves it (gemma-4 in this example). Set a Model display label if you want a friendlier name in the selector.



Models: Default model list, Fetch toggle and Model display label

3. Expand Advanced and scroll to Additional parameters. Click Add entry twice to add the two aiDAPTIV cache parameters. These are the same two keys described in Part 2:

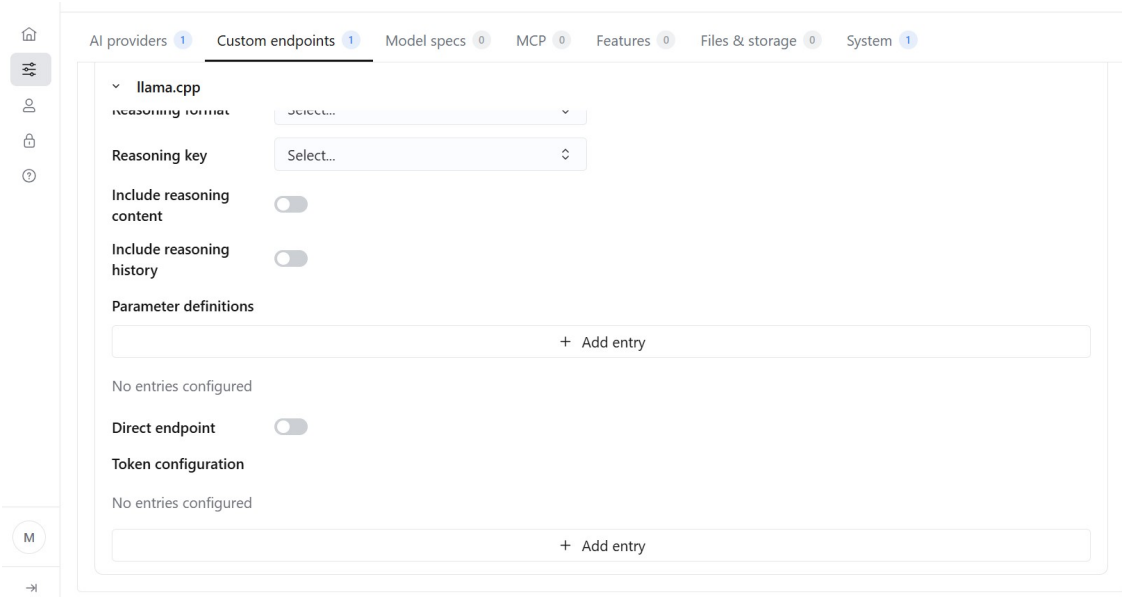


Advanced → Additional parameters: cache_prompt and offload_folder_name

Parameter	Value and type
cache_prompt	True. Set the type selector to T/F so it is sent as a true/false value, not as text.

Parameter	Value and type
offload_folder_name	Your cache folder name (benchmark-10 in this example). Set the type selector to abc (text). This is the folder you will look for in Part 5.

4. Leave every other setting (Drop parameters, Reasoning, Direct endpoint, Token configuration, etc.) at its default, then save the configuration.

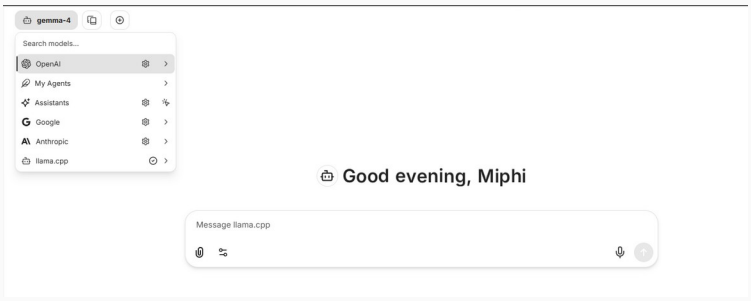


The remaining Advanced options can stay at their defaults

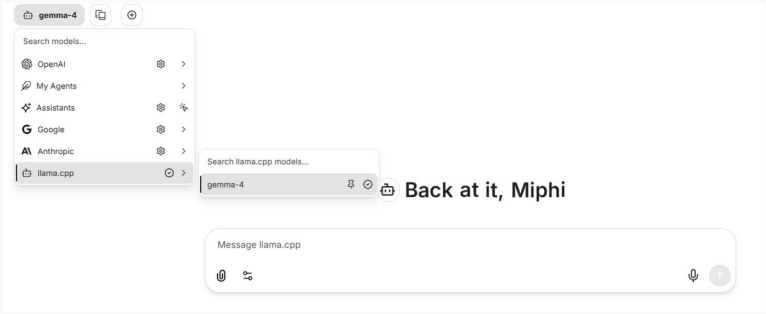
Check the endpoint in LibreChat

Once the endpoint is saved, confirm LibreChat picked it up.

1. Open LibreChat in your browser (http://localhost:3080) and, from the top of the message box, click the endpoint selector.



2. From the list of Available Endpoints, choose llama.cpp, the custom endpoint you just added.



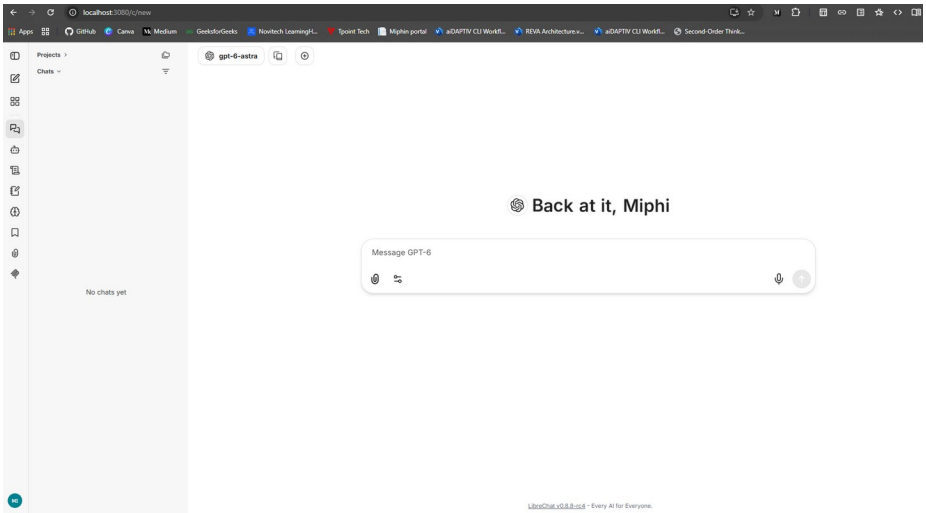
3. Confirm the fields it was saved with:

Field	What it's set to
Base URL	http://host.docker.internal:8080/v1. Use host.docker.internal, not localhost or 127.0.0.1. LibreChat's containers run on their own network and cannot see your host machine's localhost; Docker's host.docker.internal name resolves to it instead.
API Key	Leave as the placeholder value you entered or imported, unless your llama-server is configured to require one.
Models	Your model name(s), exactly as reported by your aiDAPTIV llama-server's /v1/models endpoint.

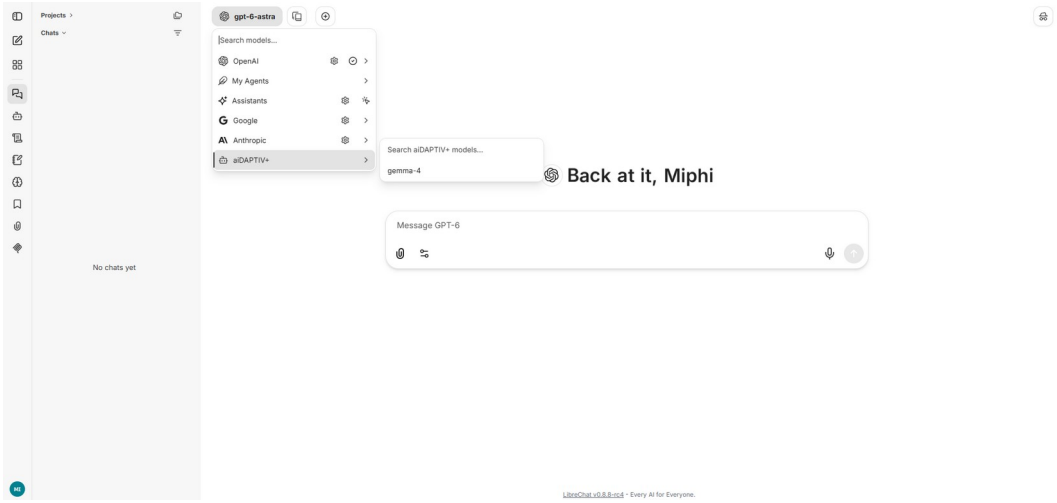
4. Select your model so it is checked and active.

Part 4: Getting Started

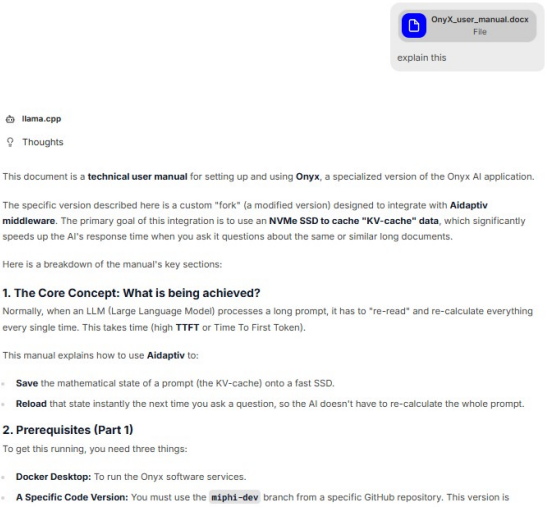
1. Open http://localhost:3080 in your browser.



2. Confirm the endpoint and model selector (top of the message box) shows the custom endpoint you added in Part 3 and the model you connected.



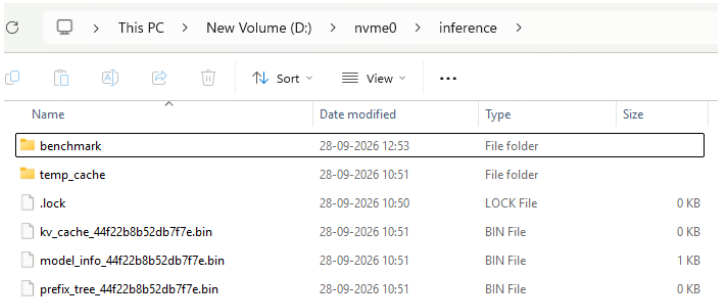
3. Type your prompt and send it. You can also attach a file first and then ask your question about it.



Part 5: Verifying the aiDAPTIV KV-Cache Offload Is Working

Check the cache folder on disk

After sending at least one prompt, browse to your aiDAPTIV NVMe cache root plus the offload_folder_name you set in Part 2 or Part 3 (for example, benchmark-10). New files confirm the KV cache is actually being written to the aiDAPTIV SSD, not just held in VRAM.



Confirm a cache hit

Ask a prompt for the first time. Its KV cache is saved to the benchmark-10 folder in aiDAPTIV. Send the same (or an overlapping) prompt again and check the llama-server log for the next request:

```
0.21.634.215 I srv  params_from_: Chat format: peg-gemma4
0.22.513.100 I slot get_availabl: id  2 | task -1 | selected slot by LRU, t_last = -1
0.22.513.181 I slot launch_slot_: id  2 | task 2 | processing task, is_child = 0
0.22.513.193 I slot update_slots: id  2 | task 2 | Hit token cnt (GPU): 0
0.22.928.411 I slot update_slots: id  2 | task 2 | Hit token cnt (DRAM): 0
0.22.928.422 I slot update_slots: id  2 | task 2 | Hit token cnt (CACHE): 12064
[MDW][Info][Runtime] Save 0 tokens to folder 'benchmark'
```

A large, non-zero Hit token cnt (CACHE) value is direct proof that those tokens were retrieved from the aiDAPTIV NVMe cache instead of being recomputed on the GPU. A closing line such as "Save 0 tokens to folder 'benchmark-10'" confirms nothing new needed to be cached because it was already there. This lower recompute workload is what produces the faster Time To First Token you'll see on repeated or overlapping prompts.

> **If your second run still shows Hit token cnt (CACHE): 0**
>
> Double-check that: cache_prompt and offload_folder_name are saved under the endpoint's Additional parameters in the Admin Panel; the prompt overlaps meaningfully with the first one; and the benchmark folder has free space on the aiDAPTIV SSD.

Part 6: Benchmark Comparison: TTFT, TPS & Cache Hit

Run 2: Cache Hit (KV Reuse)

From our benchmarking test. A previous run had already built up a token cache for this prompt; this run reused it:

```
0.22.928.411 I slot update_slots: id  2 | task 2 | Hit token cnt (DRAM): 0
0.22.928.422 I slot update_slots: id  2 | task 2 | Hit token cnt (CACHE): 12064
[MDW][Info][Runtime] Save 0 tokens to folder 'benchmark'
0.23.247.405 I slot update_slots: id  2 | task 2 | saved checkpoint to Cache Storage (n_tokens = 12064)
0.36.233.375 I slot print_timing: id  2 | task 2 | n_decoded =    100, tg =   8.13 t/s
0.36.234.394 I slot print_timing: id  3 | task 0 | n_decoded =    100, tg =   8.13 t/s
0.39.344.942 I slot print_timing: id  2 | task 2 | n_decoded =    124, tg =   8.05 t/s
0.39.346.160 I slot print_timing: id  3 | task 0 | n_decoded =    124, tg =   8.05 t/s
0.42.441.580 I slot print_timing: id  2 | task 2 | n_decoded =    149, tg =   8.05 t/s
0.42.442.684 I slot print_timing: id  3 | task 0 | n_decoded =    149, tg =   8.05 t/s
0.45.481.224 I slot print_timing: id  2 | task 2 | n_decoded =    175, tg =   8.12 t/s
0.45.482.490 I slot print_timing: id  3 | task 0 | n_decoded =    175, tg =   8.12 t/s
0.48.501.500 I slot print_timing: id  2 | task 2 | n_decoded =    200, tg =   8.14 t/s
0.48.502.575 I slot print_timing: id  3 | task 0 | n_decoded =    200, tg =   8.14 t/s
0.51.589.976 I slot print_timing: id  2 | task 2 | n_decoded =    225, tg =   8.14 t/s
0.51.590.760 I slot print_timing: id  3 | task 0 | n_decoded =    225, tg =   8.14 t/s
0.54.669.685 I slot print_timing: id  2 | task 2 | n_decoded =    250, tg =   8.13 t/s
0.54.670.667 I slot print_timing: id  3 | task 0 | n_decoded =    250, tg =   8.13 t/s
0.55.901.450 I slot update_slots: id  3 | task 0 | Multi-token MoE expert hit rate (GPU): 94.53%
0.55.901.461 I slot update_slots: id  3 | task 0 | Multi-token MoE expert hit rate (DRAM):  0.00%
[MDW][Info][Runtime] Save 304 tokens to folder 'benchmark'
0.58.189.022 I slot print_timing: id  3 | task 0 | prompt eval time =    2393.75 ms /    47 tokens (   50.93 ms per token,    19.63 tokens per second)
0.58.189.024 I slot print_timing: id  3 | task 0 |        eval time =   31964.31 ms /   260 tokens (  122.94 ms per token,     8.13 tokens per second)
0.58.189.025 I slot print_timing: id  3 | task 0 |       total time =   34358.06 ms /   307 tokens
0.58.189.027 I slot print_timing: id  3 | task 0 |     graphs reused =        255
0.58.189.101 I slot      release: id  3 | task 0 | stop processing: n_tokens = 314, truncated = 0
0.58.299.266 I slot print_timing: id  2 | task 2 | n_decoded =    261, tg =   7.59 t/s
1.01.340.103 I slot print_timing: id  2 | task 2 | n_decoded =    293, tg =   7.83 t/s
1.04.364.382 I slot print_timing: id  2 | task 2 | n_decoded =    320, tg =   7.91 t/s
1.07.381.158 I slot print_timing: id  2 | task 2 | n_decoded =    350, tg =   8.06 t/s
1.10.382.839 I slot print_timing: id  2 | task 2 | n_decoded =    381, tg =   8.20 t/s
1.13.465.324 I slot print_timing: id  2 | task 2 | n_decoded =    410, tg =   8.28 t/s
1.16.468.062 I slot print_timing: id  2 | task 2 | n_decoded =    439, tg =   8.36 t/s
1.19.474.125 I slot print_timing: id  2 | task 2 | n_decoded =    470, tg =   8.46 t/s
1.22.593.160 I slot print_timing: id  2 | task 2 | n_decoded =    500, tg =   8.52 t/s
1.25.625.962 I slot print_timing: id  2 | task 2 | n_decoded =    528, tg =   8.56 t/s
1.28.668.846 I slot print_timing: id  2 | task 2 | n_decoded =    561, tg =   8.67 t/s
1.31.687.772 I slot print_timing: id  2 | task 2 | n_decoded =    592, tg =   8.74 t/s
1.34.742.302 I slot print_timing: id  2 | task 2 | n_decoded =    621, tg =   8.77 t/s
1.37.804.449 I slot print_timing: id  2 | task 2 | n_decoded =    650, tg =   8.80 t/s
1.40.909.368 I slot print_timing: id  2 | task 2 | n_decoded =    681, tg =   8.85 t/s
1.43.939.253 I slot print_timing: id  2 | task 2 | n_decoded =    711, tg =   8.89 t/s
1.47.031.679 I slot print_timing: id  2 | task 2 | n_decoded =    742, tg =   8.93 t/s
1.50.094.214 I slot print_timing: id  2 | task 2 | n_decoded =    773, tg =   8.97 t/s
1.53.108.963 I slot print_timing: id  2 | task 2 | n_decoded =    804, tg =   9.02 t/s
1.56.196.351 I slot print_timing: id  2 | task 2 | n_decoded =    836, tg =   9.06 t/s
1.59.251.487 I slot print_timing: id  2 | task 2 | n_decoded =    867, tg =   9.10 t/s
2.02.311.992 I slot print_timing: id  2 | task 2 | n_decoded =    899, tg =   9.14 t/s
2.05.340.412 I slot print_timing: id  2 | task 2 | n_decoded =    931, tg =   9.18 t/s
2.08.409.065 I slot print_timing: id  2 | task 2 | n_decoded =    960, tg =   9.19 t/s
2.11.498.342 I slot print_timing: id  2 | task 2 | n_decoded =    989, tg =   9.19 t/s
2.14.548.511 I slot print_timing: id  2 | task 2 | n_decoded =   1017, tg =   9.19 t/s
2.17.632.537 I slot print_timing: id  2 | task 2 | n_decoded =   1046, tg =   9.20 t/s
2.20.697.671 I slot print_timing: id  2 | task 2 | n_decoded =   1075, tg =   9.21 t/s
2.23.793.849 I slot print_timing: id  2 | task 2 | n_decoded =   1104, tg =   9.21 t/s
2.26.819.376 I slot print_timing: id  2 | task 2 | n_decoded =   1134, tg =   9.23 t/s
2.28.101.004 I slot update_slots: id  2 | task 2 | Multi-token MoE expert hit rate (GPU): 94.53%
2.28.101.108 I slot update_slots: id  2 | task 2 | Multi-token MoE expert hit rate (DRAM):  0.00%
2.28.101.111 I slot update_slots: id  2 | task 2 | Single-token MoE expert hit rate (GPU): 96.02%
2.28.101.112 I slot update_slots: id  2 | task 2 | Single-token MoE expert hit rate (DRAM):  0.00%
[MDW][Info][Runtime] Save 1144 tokens to folder 'benchmark'
2.30.956.001 I slot print_timing: id  2 | task 2 | prompt eval time =    1421.16 ms /     7 tokens (  203.02 ms per token,     4.93 tokens per second)
2.30.956.005 I slot print_timing: id  2 | task 2 |        eval time =  124166.66 ms /  1147 tokens (  108.25 ms per token,     9.24 tokens per second)
2.30.956.006 I slot print_timing: id  2 | task 2 |       total time =  125587.82 ms /  1154 tokens
2.30.956.008 I slot print_timing: id  2 | task 2 |     graphs reused =       1135
2.30.957.435 I slot      release: id  2 | task 2 | stop processing: n_tokens = 13217, truncated = 0
2.30.957.475 I srv  update_slots: all slots are idle
```

TTFT: 1,421.16 ms • TPS: 9.24 tokens/sec • Hit Cache (CACHE): 12,064 tokens

Metric	Run 2, warm (cache hit)
Hit token cnt (CACHE)	12,064 tokens
TTFT (prompt eval time)	1,421.16 ms
TPS (decode / eval speed)	9.24 tokens/sec